

智能模型集成规范

针对智能模型开发过程中的训练、评估、适配等环节所需的AI资产，制定统一的平台集成规范，包括数据集、环境、算法、模型，便于进行标准化的集成、共享。

Dataset 数据集

规范

提供一张图片及md文件进行资产说明（参考 [docs/asset_description](#)，图片名为 `PROFILE.jpg`，md文件名为 `README.md`）

符合IMAGENET/VOC/COCO/YOLO-COCO标注格式

* 数据打包为 `.zip` 文件（算法读取的数据集路径为压缩包解压后的根目录）

Environment 环境

规范

以 `Docker` 镜像（`Docker`版本 ≥ 20.10 ）方式提供

环境镜像运行于 `linux` 环境

环境镜像中需安装 `Haikit` whl包 [下载](#)

镜像命名规范

*

`10.0.64.41/engine/<系统版本>-<python版本>-<算法库版本>-<框架版本>-<架构>-<计算卡>:<算法>-<用途>-<自定义版本>`

* 提供一张图片及md文件进行资产说明（参考 [docs/asset_description](#)，图片名为 `PROFILE.jpg`，md文件名为 `README.md`）

`Haikit` 说明

Haikit是一套专门为辅助算法开发及对接AI训练平台的Python SDK工具，基于标准化集成规范设计，旨在帮助算法研发人员能够更加高效地进行模型训练、评估、适配等工作，并可实现本地研发环境调试完成的算法工程可无缝集成至云端AI训练平台，对算法工程进行标准化管理与共享。

镜像制作参考 [haikit/haikit/README.md](#)

相关pip依赖见 [haikit/haikit/requirements/requirements.txt](#)

镜像命名要素 说明

| 元素 | 说明 | 是否必填 | 位置 |

|-----|-----|-----|-----|

| 系统版本（必填） | 如 `ubuntu20.04`、`centos8` | ☒ | 名称 |

| Python版本（若有） | 如 `python3.8`、`py3.11.1` | ☒ | 名称 |

| 算法库版本（若有） | 如 `cuda11.8`、`cuda8cudnn11`、`cann12.1`、`corex4.1` | ☒ | 名称 |

| 框架版本（如安装了具体框架版本，建议填写） | 如 `tf2.10`、`torch1.13`、`paddle2.4`，多个框架可以顺序补充 | ☒ |

| 名称 |

架构（非x86必填）	如 `x86`、`arm`	☒	名称
计算卡（若有且非英伟达卡，必填）	如 `a100`、`v100`、`ascend910`、`r200`、`bi150`	☒	名称
算法（如针对具体算法进行了适配，建议填写）	算法或模型名称，如 `yolov5`、`resnet50`	☒	标签
用途（可选）	如 `train`、`infer`	☒	标签
自定义版本	如 `v1.0.0`、`v20250405`、`beta1`、`zhangsan`	☒	标签

- **前缀固定：** `10.0.64.41/engine/` 为固定前缀，用于标识镜像来源和仓库位置。
- **必填项：** 系统、Python（若有）、算法库（若有）、架构（非x86必填）、计算卡（若有且非英伟达卡，必填）。
- **可选项：** 框架（如安装了具体框架版本，建议填写）、算法（如针对具体算法进行了适配，建议填写）、用途、自定义版本。
- **算法库版本说明：**
 - 若有，则必填，关系到可安装框架的版本。
 - 使用通用命名方式，如 `cuda11.8`、`cuda8cudnn11`、`cann12.1`、`corex4.1`。
 - 用于标识所依赖的算法库版本。
- **架构区分：**
 - 非 x86 架构必填，如 `arm`。
- **计算卡说明：**
 - 若有且非英伟达卡，必填。
 - 用于标识镜像所适配的 GPU 或 AI 加速卡型号，如 `a100`、`v100`、`ascend910`、`r200`、`bi150`。
- **自定义版本命名建议：**
 - 推荐使用语义化版本（如 `v1.0.0`）
 - 或使用时间戳（如 `v20250405`）
 - 或项目号（如 `bxxxx`）
 - 或所属人（如 `zzz`）

镜像命名示例

- 必填项齐全

- 示例 1：通用 x86 架构
`10.0.64.41/engine/ubuntu20.04-python3.8-cuda11.9:v1.0.0`
- 示例 2：arm 架构
`10.0.64.41/engine/ubuntu20.04-python3.11-cann12.1-arm-ascend910:v1.0.0`

• 选填齐全

包含必填项 + 可选项（推荐）

- 示例 1：通用 x86 架构

`10.0.64.41/engine/ubuntu20.04-python3.8-cuda11.9-tf2.10-a100:yolov5-train-v1.0.0-zyl`

- 示例 2：arm 架构

```
10.0.64.41/engine/ubuntu20.04-python3.11-corex4.1-paddle2.4-arm-bi150:resnet50-train-  
v1.0.0-jsx
```

Algorithm 算法

规范

以 **算法包** 形式配套 **环境镜像** 一同提供

算法包符合 **haikit** 规范

- * 算法包中需放置 **config.yaml**（名称可以自定义，可配置多个）文件，用于描述任务的配置信息（执行脚本路径，超参数、环境变量、计算资源等）
- * 算法包中需包含训练（train）/评估（evaluate）/适配（adapt）/服务（service）任务相关的启动脚本及所有依赖文件
- * 算法包中各任务实际运行的py文件中需增加相关的 **haikit** 接口
- * 算法工程应具备 **可移植性**，即不依赖于绝对文件路径，不能因工程部署路径变化而导致执行失败（例如，代码中有 `sys.path.append('/mnt/sde/yolox/')`，则代码与路径强耦合了，若有这种需求，需修改为动态获取工程路径）
- * 提供一张**图片**及**md文件**进行资产说明（参考 **docs/asset_description**，图片名为 **PROFILE.jpg**，md文件名为 **README.md**）

可参考 **alg-demo** 进行算法工程修改与调试

- 参考 **train.sh/eval.sh/adapt.sh** 生成启动脚本（增加环境变量，运行原本的启动脚本）
- 参考 **train.py/eval.py/adapt.py** 中 **TODO** 进行当前算法工程的修改
- 修改完成后可基于安装有haikit的镜像进行本地调试，调试通过则该工程可直接迁移至平台

算法包目录示例

```
-- alg_demo  
-- train.sh（支持.sh文件，名称与config.yaml中对应）  
-- eval.sh  
-- convert.sh  
-- service.sh（建议基于FastAPI及gradio进行服务封装）  
-- config.yaml
```

config.yaml 格式示例

```
train:  
  # 代码根目录（必选），"/path/to/codedir"  
  code_dir: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1"  
  # 启动容器时执行的脚本（必选），"/path/to/your_script.sh"  
  script_path: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1/alg-demo/train.sh"  
  # 启动容器时执行的命令（可选），"your_command"  
  # command: ["bash", "train.sh"]  
  # 超参数配置（可选，用于暴露可供调参的超参数，建议配置）  
  parameters:  
    # param1: value1  
    lr: 0.001  
    epochs: 20
```

```

# 环境变量登记（可选，运行所需环境变量，平台运行时加载，本地调试时不生效）
environment_variables:
    # ENV_VAR1: "value1"
    TZ: "Asia/Shanghai"
# 计算资源登记（必选，建议登记为任务执行所需的最小资源量，平台运行时加载，本地调试时不生效）
resources:
    cpu: 1
    gpu: 0
    memory: 5
# 数据路径配置（可选），"/path/to/local/data"
# data_dir: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1/haikit-code/dataset-demo"
# 模型路径配置（可选），"/path/to/local/model_pkg"（模型文件包需符合haikit目录结构规范，用于加载模型）
# model_dir: "/mnt/hgfs/shareplace/workplace/sdk_dev/asset_demo/a-demo-model"
# 模型用途（可选，配置模型路径时则为必选），可选项为“pretrain”（预训练）和“resume”（断点续训）
# model_usage: "pretrain"
# 任务结果保存路径（必选，为本地调试时指标、模型保存的根目录），"/path/to/savedir"
save_dir: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1/alg-demo/train_result"

evaluate:
    # 代码根目录，"/path/to/codedir"
    code_dir: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1"
    # 启动容器时执行的脚本，"/path/to/your_script.sh"
    script_path: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1/alg-demo/eval.sh"
    # 启动容器时执行的命令，"your_command"
    # command: ["bash", "eval.sh"]
    # 超参数配置（可选）
    parameters:
        iou_threshold: 0.5
        score_threshold: 0.7
# 环境变量登记（可选，运行所需环境变量，平台运行时加载，本地调试时不生效）
environment_variables:
    # ENV_VAR1: "value1"
    TZ: "Asia/Shanghai"
# 计算资源登记（必选，建议登记为任务执行所需的最小资源量，平台运行时加载，本地调试时不生效）
resources:
    cpu: 1
    gpu: 0
    memory: 5
# 数据路径配置，"/path/to/local/data"
data_dir: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1/haikit-code/dataset-demo"
# 模型路径配置（可选），"/path/to/local/model_pkg"（模型文件包需符合haikit目录结构规范，用于加载模型）
model_dir: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1/alg-demo/train_result"
# 任务结果保存路径（必选，为本地调试时指标、模型保存的根目录），"/path/to/savedir"
save_dir: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1/alg-demo/eval_result"

adapt:
    # 代码根目录，"/path/to/codedir"
    code_dir: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1"
    # 启动容器时执行的脚本，"/path/to/your_script.sh"
    script_path: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1/alg-demo/convert.sh"
    # 启动容器时执行的命令，"your_command"

```

```

# command: ["bash", "convert.sh"]
# 指定适配类型（可选，配置后便于任务类型识别归类），可配置类型如下：
# - distill（蒸馏）
# - prune（剪枝）
# - lightweight（轻量化）
# - format_transfer（格式转换）
adapt_type: "lightweight"
# 超参数配置（可选）
parameters:
  # param1: value1
  target_model: "onnx"
# 环境变量登记（可选，运行所需环境变量，平台运行时加载，本地调试时不生效）
environment_variables:
  # ENV_VAR1: "value1"
  TZ: "Asia/Shanghai"
# 计算资源登记（必选，建议登记为任务执行所需的最小资源量，平台运行时加载，本地调试时不生效）
resources:
  cpu: 1
  gpu: 0
  memory: 5
# 数据路径配置（可选），"/path/to/local/data"
data_dir: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1/haikit-code/dataset-demo"
# 模型路径配置（可选），"/path/to/local/model_pkg"（模型文件包需符合haikit目录结构规范，用于加载模型）
model_dir: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1/alg-demo/train_result"
# 任务结果保存路径（必选，为本地调试时指标、模型保存的根目录），"/path/to/savedir"
save_dir: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1/alg-demo/adapt_result"

service:
# 代码根目录（可选），"/path/to/codedir"
code_dir: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1"
# 启动容器时执行的脚本（可选），"/path/to/your_script.sh"
script_path: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1/alg-demo/service.sh"
# 启动容器时执行的命令（可选，若未配置code_dir和script_path，则必选），"your_command"
# command: ["bash", "service.sh"]
# 环境变量登记（可选，运行所需环境变量，平台运行时加载，本地调试时不生效）
environment_variables:
  # ENV_VAR1: "value1"
  TZ: "Asia/Shanghai"
# 计算资源登记（必选，建议登记为任务执行所需的最小资源量，平台运行时加载，本地调试时不生效）
resources:
  cpu: 2
  gpu: 0
  memory: 4
# 模型路径配置（可选），"/path/to/local/model_pkg"（模型文件包需符合haikit目录结构规范，用于加载模型）
model_dir: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1/alg-demo/train_result"
# 任务结果保存路径（可选，为本地调试时指标保存的根目录），"/path/to/savedir"
# save_dir: "/nfs/zyl/workplace/hai/gitlab_nuwa/nuwa/algorithm/1/alg-demo/adapt_result"
# 监听接口（必选，可配置多个，不能重名），用于访问服务
ports:
- name: port1

```

```
port: 80
- name: port2
port: 81
```

注，若为分布式训练任务，则需要对 `resources` 内容进行如下格式替换

```
# 分布式计算资源配置
resources:
# 支持 ddp(pytorch)、ps-worker(tensorflow)两种模式
distributed_mode: "ddp"
master:
replicas: 1
cpu: 1
gpu: 0      # 影响环境变量gpu_num
memory: 5
worker:
replicas: 1
cpu: 1
gpu: 0      # 影响环境变量gpu_num
memory: 5
```

🌟Model 模型

规范

提供一张图片及md文件进行资产说明（参考 `docs/asset_description`，图片名为 `PROFILE.jpg`，md文件名为 `README.md`）

以 `模型包` 形式提供

模型包中须包含 `model` 文件夹，放置模型文件，如 `model.pt`

模型包中须包含 `config` 文件夹，放置除模型文件外的其他文件，如模型信息文件 `model_info.json`（若无其他非模型文件时，可创建空文件夹）

* 模型包打包为 `.zip` 文件（算法读取的模型路径为压缩包解压后的根目录）

模型包目录示例

```
-- model_demo
-- config
-- model_info.json（包含模型生成相关信息、模型指标信息）
-- model（包含具体模型文件）
```

`model_info.json` 格式示例

```
{
  "version_info": {
    "train_config": {
```

```
    "parameters": {
      "epochs": 11,
      "lr": 0.001,
      "batch_size": 16
    },
    "environment_variables": {
      "ENV_VAR1": "value1",
      "ENV_VAR2": "value2"
    },
    "resources": {
      "cpu": 0.5,
      "gpu": 0,
      "memory": 0.5
    },
  },
  "evaluate_config": {
    "parameters": {
      "iou_threshold": 0.5,
      "score_threshold": 0.7
    },
    "environment_variables": {
      "ENV_VAR1": "value1",
      "ENV_VAR2": "value2"
    },
    "resources": {
      "cpu": 0.5,
      "gpu": 0,
      "memory": 0.5
    },
  },
  "adapt_config": {
    "parameters": {
      "target_model": "onnx"
    },
    "environment_variables": {
      "ENV_VAR1": "value1",
      "ENV_VAR2": "value2"
    },
    "resources": {
      "cpu": 0.5,
      "gpu": 0,
      "memory": 0.5
    },
  },
  "service_config": {
    "parameters": {
      "target_model": "onnx"
    },
    "environment_variables": {
      "ENV_VAR1": "value1",
      "ENV_VAR2": "value2"
    },
  },
}
```

```
    },
    "resources": {
      "cpu": 0.5,
      "gpu": 0,
      "memory": 0.5
    },
    "ports": [{"name": "port1", "nodePort": 1322}],
  },
  "metrics": {
    "train": {
      "train": {
        "loss": 0.1,
        "map": 0.9,
        "recall": 0.9,
        "precision": 0.9
      }
    },
    "evaluate": {
      "evaluate": {
        "map": 0.6,
        "recall": 0.6,
        "precision": 0.9
      }
    },
    "adapt": {
      "adapt": {
        "map": 0.3,
        "recall": 0.4,
        "precision": 0.6,
        "fps": 15
      }
    }
  },
},
}
```